

Solver Specification:

highOrderElectroActivationFoamExplicit

A high-order, explicit forward-Euler monodomain solver in OpenFOAM with CFL-bounded time step

Pablo

May 21, 2026

Abstract

This report specifies the algorithms implemented in the OpenFOAM solver:

highOrderElectroActivationFoamExplicit.

The solver advances the monodomain equations of cardiac electrophysiology coupled to the ten Tusscher–Noble–Noble–Panfilov (TNNP, 2004) ionic cell model. The diffusion operator is discretised with the cell-centred finite-volume framework of OpenFOAM, optionally upgraded to a Local Regression Estimators (LRE) high-order reconstruction; the ionic source is evaluated either at cell centres or at cell quadrature points; time integration is the explicit forward-Euler scheme, with a stable time step bounded by the parabolic CFL condition computed at run time from the local diffusivity and the characteristic cell size. A staggered operator-split structures the per-step work: the algebraic ionic current at t^n is updated first, then the diffusion Laplacian is assembled, the voltage is advanced by an Euler step, and finally the TNNP cell ODEs are integrated from t^n to t^{n+1} using the just-updated voltage. The per-cell ionic ODE uses OpenFOAM’s adaptive RKF45 integrator with configurable tolerances; the TNNP cell model is wrapped in a phase-aware `protectState` guard that enforces physical bounds before every algebraic evaluation; activation times are recorded by linear interpolation across the first threshold crossing and benchmark points are sampled by inverse-distance weighting. The report also discusses how the CFL-bounded explicit scheme behaves on the 2D and 3D Niederer-style benchmark runs: in particular, how the reported *requested* Δt relates to the *effective* Δt the solver actually uses after the CFL clamp. All algorithmic parameters are exposed through the `constant/spatialIntegrationProperties` and `constant/timeIntegrationProperties` dictionaries, and the companion Python driver `run_convergence.py` generates these dictionaries from a single configuration file. Section 10 collects measured P8 activation times, wall-clock cost and peak memory from a 2D Niederer-style sweep and a sparser 3D Niederer-style sweep.

Contents

1	Notation	3
2	Governing equations	3
2.1	Monodomain PDE	3
2.2	Ionic ODE	3
2.3	External stimulus	4
3	Time discretisation: forward Euler with parabolic CFL bound	4
3.1	Stable time step	4
4	Spatial discretisation	5
4.1	Standard orthogonal FV Laplacian	5
4.2	High-order LRE diffusion	5
4.3	Ionic source quadrature	5

5	Per-step operator splitting	6
6	Ionic ODE integration	6
7	TNNP numerical protection	7
7.1	Master switch and configuration	7
7.2	Phases at which the guard fires	7
7.3	Corrections and their order	7
7.4	Interaction with the RKF45 step controller	8
7.5	What the guard does not protect	8
7.6	Logging and accounting	9
7.7	Defaults vs. recommended settings	9
8	Activation tracking and benchmarks	9
9	Driver script: <code>run_convergence.py</code>	10
9.1	Geometry and parameter sweep	10
9.2	Inner ODE integrator (RKF45)	10
9.3	Effective Δt reported in result tables	10
9.4	Post-processing and run-control	10
10	Niederer-style benchmark results	11
10.1	2D Niederer-style sweep	11
10.2	P8 activation time at the requested $\Delta t = 1.0 \cdot 10^{-2}$ s	11
10.3	Wall-clock cost per configuration (minutes)	12
10.4	Peak resident set size per configuration (MiB)	12
10.4.1	Discussion of the 2D results	13
10.5	3D Niederer-style sweep	13
10.6	P8 activation time at the requested Δt	14
10.7	Wall-clock cost per configuration (minutes)	14
10.8	Peak resident set size per configuration (MiB)	15
10.8.1	Discussion of the 3D results	15
10.9	Conclusions on the 2D/3D study	16
A	Dimension table	16
B	TNNP rate expressions (verbatim)	16

1 Notation

Symbol	Meaning	Units
$\Omega \subset \mathbb{R}^d$	Spatial domain ($d \in \{2, 3\}$)	m
$\partial\Omega$	Insulated boundary (zero-flux Neumann)	—
$t \in (0, T]$	Time	s
$V_m(\mathbf{x}, t)$	Transmembrane voltage	V
$\mathbf{s}(\mathbf{x}, t) \in \mathbb{R}^{17}$	TNNP ionic states	mixed
$I_{\text{ion}}(V_m, \mathbf{s})$	Total ionic current density	V s^{-1}
$I_{\text{ext}}(\mathbf{x}, t)$	External stimulus current density	A m^{-3}
χ	Membrane surface-to-volume ratio	m^{-1}
C_m	Membrane capacitance per unit area	F m^{-2}
$\mathbf{D}(\mathbf{x})$	Bulk conductivity tensor	S m^{-1}
Δt	Outer time-step length (effective)	s
Δt_{req}	Requested outer time step (<code>controlDict.deltaT</code>)	s
Δt_{CFL}	CFL-stable time step computed at run-time	s
Δx	Characteristic cell size	m
CFL	Safety coefficient of the parabolic CFL bound	—
N_c	Number of mesh cells	—
$\mathbf{V}^n \in \mathbb{R}^{N_c}$	Vector of cell-averaged V_m at time t^n	V

2 Governing equations

2.1 Monodomain PDE

The bulk voltage is propagated by the standard monodomain reaction–diffusion equation [1, 2]:

$$\chi C_m \frac{\partial V_m}{\partial t} = \nabla \cdot (\mathbf{D} \nabla V_m) - I_{\text{ion}}(V_m, \mathbf{s}) + I_{\text{ext}} \quad \text{in } \Omega \times (0, T], \quad (1)$$

subject to insulated boundaries $\mathbf{n} \cdot (\mathbf{D} \nabla V_m) = 0$ on $\partial\Omega \times (0, T]$, and initial condition $V_m(\mathbf{x}, 0) = -84 \text{ mV}$. The conductivity tensor $\mathbf{D}$ is symmetric positive-definite and either uniform or read as a `volTensorField` from disk.

Dividing (1) by χC_m gives the form actually used inside the solver:

$$\frac{\partial V_m}{\partial t} = \mathcal{L} V_m + s(V_m, \mathbf{s}), \quad \mathcal{L} = \frac{1}{\chi C_m} \nabla \cdot (\mathbf{D} \nabla \cdot), \quad s = -\frac{I_{\text{ion}}}{\chi C_m} + \frac{I_{\text{ext}}}{\chi C_m}. \quad (2)$$

2.2 Ionic ODE

At every spatial sampling point the TNNP state vector $\mathbf{s} \in \mathbb{R}^{17}$ evolves according to

$$\frac{d\mathbf{s}}{dt} = \mathbf{f}_{\text{TNNP}}(V_m, \mathbf{s}; \text{tissue}), \quad \mathbf{s}(0) = \mathbf{s}_0, \quad (3)$$

where `tissue` selects `epicardialCells`, `mCells` or `endocardialCells`. The 17 tracked states are

$$\mathbf{s} = (V, K_i, Na_i, Ca_i, Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g, Ca_{SR}).$$

The total ionic current returned to the PDE is `Iion_cm` of TNNP [1]; the full rate-equation set is reproduced in Appendix B for reference.

Units and sign convention. The field `Iion_cm` returned by the TNNP wrapper is the per-capacitance ionic current,

$$\text{Iion_cm} = \frac{I_{\text{ion}}^{(\text{memb})}}{C_m} \equiv \frac{1}{\chi C_m} (\chi I_{\text{ion}}^{(\text{memb})}),$$

expressed in V/s, with the electrophysiological convention *outward (cation efflux) current positive*. Equation (2) therefore adds $-I_{\text{ion_cm}}$ to the PDE source; the external stimulus `externalStimulusCurrent` is stored in A/m³ and divided by χC_m inside the solver (see `createFields.H` and the `rhsVm` assembly in the main solver loop). Throughout the rest of this report the symbol I_{ion} is used as a shorthand for `Iion_cm`; no further rescaling is applied between the cell model and the explicit update.

2.3 External stimulus

$$I_{\text{ext}}(\mathbf{x}, t) = \begin{cases} I_0 & \mathbf{x} \in \Omega_{\text{stim}}^{(b)}, \quad t \in [t_0^{(b)}, t_0^{(b)} + \tau], \\ 0 & \text{otherwise,} \end{cases} \quad (4)$$

with Niederer-benchmark defaults $I_0 = 50\,000 \text{ A m}^{-3}$ and $\tau = 2 \text{ ms}$.

3 Time discretisation: forward Euler with parabolic CFL bound

The solver advances the semi-discrete equation (2) by a single forward-Euler step on the whole right-hand side:

$$\mathbf{V}^{n+1} = \mathbf{V}^n + \Delta t \left[\mathbf{L} \mathbf{V}^n - I_{\text{ion}}^n + I_{\text{ext}}^n / (\chi C_m) \right], \quad (5)$$

with $\mathbf{L}$ the discrete operator of $\mathcal{L}$ (Section 4). No implicit matrix is assembled and no linear system is solved.

3.1 Stable time step

The diffusion half of (2) is parabolic; the corresponding explicit stability bound is

$$\Delta t_{\text{CFL}} = \text{CFL} \frac{\Delta x^2}{d D_{\text{eff,max}}}, \quad D_{\text{eff,max}} = \frac{\max(D_{xx}, D_{yy}, D_{zz})}{\chi C_m}, \quad (6)$$

where d is the active geometric dimension (`mesh.nGeometricD()` in 1/2/3D), and the characteristic cell size is

$$\Delta x = \left(\frac{\text{bbox active volume}}{N_c} \right)^{1/d} (\text{characteristicDx}). \quad (7)$$

For a uniform Cartesian mesh (7) returns the standard cell edge. The bound (6) uses the maximum of the diagonal conductivity components — sufficient for diagonal-dominated conductivity tensors of the Niederer-style benchmarks; for strongly anisotropic or non-diagonal $\mathbf{D}$ the bound is conservative.

At solver start the effective step is set to

$$\Delta t = \min(\Delta t_{\text{req}}, \Delta t_{\text{CFL}}), \quad (8)$$

where Δt_{req} is the value of `controlDict.deltaT`; the value is then written back into the running time database with `runTime.setDeltaT(dt)`. The driver script `run_convergence.py` records both quantities in the results tables (column `dt_requested` versus `dt_expectedSolver`); see Section 10.

Implication. For the configurations reported in Section 10 the requested $\Delta t = 10^{-2} \text{ s}$ is many orders of magnitude above $\Delta x^2 / (d D_{\text{eff,max}})$; the solver therefore always runs at $\Delta t = \Delta t_{\text{CFL}}$ and the *requested* value only sets an upper bound on the step. The column `dt_expectedSolver` in the result tables is the actual step. Tightening or loosening `controlDict.deltaT` below the CFL value is the only way to override the bound, at the obvious cost of stability.

4 Spatial discretisation

4.1 Standard orthogonal FV Laplacian

When `useHighOrder.Vm = false` (or in 1D), the diffusion is evaluated by OpenFOAM's built-in operator

$$\text{LV} \approx \frac{1}{\chi C_m} \text{fvc::laplacian}(\text{conductivity}, \text{Vm}), \quad (9)$$

i.e. the classical orthogonal 5/7-point stencil with linear face interpolation and the corrected `snGrad`. `zeroGradient` and `empty` patches contribute zero flux; `fixedValue` (or the custom `fixedVoltage`) patches contribute the one-sided owner gradient via the standard FV path.

4.2 High-order LRE diffusion

When `useHighOrder.Vm = true` and $d > 1$, the Laplacian is assembled by an explicit face-quadrature integration (`computeHighOrderLaplacian` in the main solver):

$$\Phi_f = \sum_{q \in \mathcal{Q}_f} w_q |\mathbf{S}_f| \mathbf{n}_f \cdot \mathbf{D}_{\text{owner}(f)} \cdot \nabla_h V_m(\mathbf{x}_{fq}), \quad (10)$$

with face quadrature points $\mathbf{x}_{fq}$ and weights w_q provided by the LRE library [4]. The cell contribution to the Laplacian is then

$$(\text{LV})_i = \frac{1}{\chi C_m} \frac{1}{|K_i|} \sum_{f \in \partial K_i} s_{if} \Phi_f, \quad (11)$$

with $s_{if} = \pm 1$ for owner/neighbour orientation. The cell-loop divergence is taken with `fvc::div(fluxVm_HO)` after the face fluxes are written.

Boundary faces in the high-order branch. For boundary faces, the LRE face quadrature still evaluates a reconstructed gradient, but the flux is enforced as follows: `zeroGradient` and `empty` patches set $\Phi_f \equiv 0$ (true Neumann zero-flux, even though the LRE stencil could return a non-zero gradient otherwise); `fixedValue` and `fixedVoltage` patches use the LRE-reconstructed gradient on the owner side. The high-order LRE stencil is therefore strictly internal in the Neumann-only case used in the 2D/3D benchmarks reported here, and the formal spatial order is expected to drop to that of the standard branch in the first layer of cells adjacent to a non-empty patch.

4.3 Ionic source quadrature

If `useHighOrder.Iion = false`, the cell-model is integrated at cell centres:

$$\overline{I_{\text{ion}i}} = I_{\text{ion}}(V_{m,i}, \mathbf{s}_i). \quad (12)$$

If `useHighOrder.Iion = true` the TNNP states are evolved at the cell quadrature points $\mathbf{x}_{iq}$ provided by the LRE library: V_m is reconstructed at each point by a Taylor expansion (`reconstructVmAtIionIntegrationPoints`).

$$V_{m,iq} = V_{m,i} + \nabla V_{m,i} \cdot (\mathbf{x}_{iq} - \mathbf{x}_i) + \frac{1}{2} (\mathbf{x}_{iq} - \mathbf{x}_i)^T \mathbf{H}_i (\mathbf{x}_{iq} - \mathbf{x}_i) + \frac{1}{6} \mathbf{T}_i^{(3)} : (\mathbf{x}_{iq} - \mathbf{x}_i)^{\otimes 3}, \quad (13)$$

where the Hessian and third-order term are included only if the LRE order on V_m is ≥ 2 or ≥ 3 respectively. When `useHighOrder.Vm = false`, only the linear term is used and the gradient comes from `fvc::grad`. The ionic ODE is then advanced at each quadrature point against this frozen $V_{m,iq}$, and the cell-averaged source is the quadrature

$$\overline{I_{\text{ion}i}} = \frac{\sum_{q \in \mathcal{Q}_i} w_q I_{\text{ion}}(V_{m,iq}, \mathbf{s}_{iq})}{\sum_{q \in \mathcal{Q}_i} w_q} \quad (\text{averageIionIntegrationPointsToCells}). \quad (14)$$

5 Per-step operator splitting

The solver uses a staggered operator-split per outer step. For $t^n \rightarrow t^{n+1} = t^n + \Delta t$:

Algorithm 1 Per-step flow of `highOrderElectroActivationFoamExplicit`.

- 1: Save V^n from the current field.
- 2: Apply stimulus to I_{ext} at t^n .
- 3: **First ionic call (algebraic)**. Call `ionicModel->calculateCurrent` at the current state $\mathbf{s}^n$ and voltage V^n to obtain I_{ion}^n . This is an *algebraic* evaluation; the state vector is *not* advanced here.
- 4: Build LV^n (standard FV or HO LRE, Section 4).
- 5: Update voltage by forward Euler:

$$V^{n+1} = V^n + \Delta t [LV^n - I_{\text{ion}}^n + I_{\text{ext}}^n/(\chi C_m)].$$

- 6: **Second ionic call (ODE)**. Call `ionicModel->solveODE` from t^n to t^{n+1} with voltage V^{n+1} *frozen*, advancing $\mathbf{s}^n \rightarrow \mathbf{s}^{n+1}$. If `useHighOrder_Iion`, the ODE is advanced at the cell quadrature points with reconstructed $V_{m,iq}^{n+1}$, and the cell-averaged I_{ion}^{n+1} is written via Eq. (14).
 - 7: Update activation-time field (Section 8).
 - 8: If `stopAfterPointActivation`: check the stop cell.
 - 9: Write fields if `runTime.outputTime`.
-

Order of accuracy in time. The split is first-order accurate: the voltage update (5) is plain forward Euler, and the cell-model is integrated against the end-of-step voltage rather than a time-centred value. Restoring a second-order overall scheme would require either a Strang split with half-steps on the ionic ODE, or replacing the forward Euler by a Heun-/RK2-type evaluation; neither is implemented.

Why two distinct ionic calls. The first call (`calculateCurrent`) evaluates the algebraic I_{ion} at the current state; this is the source seen by the voltage update (5). The second call (`solveODE`) advances the gating and concentration ODEs to t^{n+1} and *also* returns I_{ion}^{n+1} , which is stored in the field `ionicCurrent` for output and used by the next step. The two calls share no internal state — the first one does not pre-advance any gate — so the order is “algebraic source first, then ODE update with the new voltage”.

6 Ionic ODE integration

For each cell or cell QP, the TNNP ODE (3) is advanced by the OpenFOAM adaptive integrator selected via the `solver` key in `timeIntegrationProperties`. Default RKF45 [3]. The keys read from that file are:

Key	Meaning	Default
<code>solver</code>	ODE integrator	RKF45
<code>absTol</code>	Absolute tolerance per state	10^{-9}
<code>relTol</code>	Relative tolerance per state	10^{-6}
<code>maxSteps</code>	Cap on sub-steps per outer call	10^4
<code>initialODEStep</code>	Initial sub-step (ms)	10^{-5}
<code>CFL</code>	Diffusion CFL coefficient (Eq. (6))	0.1

The outer CFL coefficient sits in the same dictionary purely for convenience; functionally it is a time-stepping parameter (Section 3.1) and not an ODE-integrator parameter.

7 TNNP numerical protection

The TNNP 2004 cell model contains several mathematical features that become numerical traps when the state variables are driven outside their physiological window by the outer solver: gate variables that overshoot $[0, 1]$ make the rate-balance equations stiff or unphysical; intracellular concentrations that touch zero force $\log(\cdot)$ / $\sqrt{\cdot}$ / $\text{pow}(\cdot)$ to overflow; very large $|V|$ saturates the $\exp(\cdot)$ tails of the α/β rate constants; and the L -type calcium current i_{CaL} contains a removable singularity at $V = 0$ (a $V/(\exp(V/RT) - 1)$ factor, see Eq. (30) in Appendix B). The TNNP wrapper provides a guard, `protectState(state, t, qp, phase)`, whose job is to keep the state vector inside that physiological window at the boundaries of every external call into the cell-model kernel. The guard is intentionally minimal: it does *not* fix the cell model, it only prevents one bad time-step from cascading into NaN/Inf and corrupting the rest of the simulation.

7.1 Master switch and configuration

Key	Type	Default
<code>TNNPNumericalProtection</code>	Switch	<code>false</code>
<code>TNNPCLampGates</code>	Switch	<code>true</code>
<code>TNNPCLampVmForRates</code>	Switch	<code>true</code>
<code>TNNPLogNumericalProtection</code>	Switch	<code>true</code>
<code>TNNPConcentrationFloor</code>	scalar (mM)	10^{-12}
<code>TNNPVMInForRates</code>	scalar (mV)	-200
<code>TNNPVMMaxForRates</code>	scalar (mV)	$+100$
<code>TNNPVoltageZeroEps</code>	scalar (mV)	10^{-8}
<code>TNNPMaxProtectionLogEntries</code>	label	10^6

The master switch is `TNNPNumericalProtection`. When it is `false`, `protectState` returns immediately and *none* of the sub-switches have any effect; in particular, `TNNPCLampGates` and `TNNPCLampVmForRates` default to `true` but are inert without the master switch. The default configuration is therefore *all protection off*; production runs typically need to set `TNNPNumericalProtection true` explicitly.

7.2 Phases at which the guard fires

The wrapper invokes the guard at four phases, distinguished by a string tag passed to `protectState` (used only for logging / accounting; see §7.6):

Phase tag	Call site	Acts on
<code>calculateCurrent</code>	entry of <code>calculateCurrent</code>	the state vector passed in for the algebraic I
<code>solveODE_start</code>	entry of <code>solveODE</code>	the integrator's working state vector at t^n (s)
<code>solveODE_end</code>	exit of <code>solveODE</code>	the integrator's working state vector at t^{n+1}
<code>derivatives</code>	inside <code>derivatives()</code> , every RKF45 sub-step	a <i>copy</i> of the state vector (does not mutate y)

All four sites are gated by the single `TNNPNumericalProtection` flag. The `derivatives` site has one important asymmetry: it copies the integrator state and protects the copy, then feeds the copy into the rate kernel `TNNPcomputeRates`. The integrator's working vector y is never modified mid-step, so the protection only changes the value of the rates RKF45 sees, not the state it integrates.

7.3 Corrections and their order

When fired, `protectState` applies the following corrections, in this order, to the (possibly copied) state vector:

1. **Non-finite V :** if `state[V]` is NaN or Inf, it is reset to -86.2 mV (TNNP resting potential, by construction inside `[vmMin, vmMax]`).
2. **Voltage clamp for rates** (if `TNNPCLampVmForRates`): V is clamped to `[TNNPVMinForRates, TNNPVMaxForRates]`. After step 1 this step is a no-op; it only fires when V was finite but outside the rate window.
3. **Avoid i_{CaL} singularity:** if $|V| < \text{TNNPVoltageZeroEps}$, V is pushed to $\pm \text{TNNPVoltageZeroEps}$ (sign preserved). This removes the $V/(\exp(V/RT) - 1)$ removable singularity in i_{CaL} at the symbolic level. Steps 1–2 may have moved V off zero already; this is the safety net for the case where V is finite, inside the rate window, but right on zero.
4. **Concentration floor:** each of K_i, Na_i, Ca_i, Ca_{SR} is replaced by `TNNPConcentrationFloor` (10^{-12} mM) if it is non-finite or below the floor. This prevents $\log(\cdot)$ and rational-exponent operations on near-zero concentrations from overflowing.
5. **Gate clamp** (if `TNNPCLampGates`): the twelve gating variables $Xr_1, Xr_2, Xs, m, h, j, d, f, fCa, s, r, g$ are clamped to $[0, 1]$ (with non-finite values pulled to 0). This is the most frequent correction in practice because adaptive ODE sub-stepping can overshoot the bounds by $\mathcal{O}(\text{relTol})$ between t^n and t^{n+1} .

Each step is idempotent and operates on the already-corrected state, so a single `protectState` call is enough — there is no inner fixed-point iteration. The ionic currents are computed from the corrected state after step 5.

7.4 Interaction with the RKF45 step controller

The `derivatives` call site introduces *non-smoothness* in the right-hand side seen by RKF45: clamping is C^0 at best (piecewise linear in the offending component), and the gate clamp is a saturation that flattens the Jacobian once the gate touches its bound. The RKF45 embedded error estimator implicitly assumes C^{q+1} smoothness of the RHS over the sub-step, so firing the guard inside `derivatives()` can inflate the local error estimate around clamp boundaries and trigger spurious sub-stepping. Because the integrator’s working state y is *not* mutated (§7.2), the practical effect is bounded: only the rates are perturbed, the state recovers as soon as the active gate leaves the clamp. Even so, a high count of `phase=derivatives` corrections in the summary file is a diagnostic signal that the cell ODE is being driven outside its valid regime — usually by a too-large Δt (note: the outer Δt here is already bounded by CFL, so this is more likely a transient in V_m that the explicit diffusion solver could not follow). The recommended response is to investigate the upstream cause; the `derivatives`-phase protection is a safety net, not a correction mechanism.

7.5 What the guard does not protect

The five corrections above address NaN/Inf propagation, gate excursion outside $[0, 1]$, the $V = 0$ singularity of i_{CaL} , and concentration underflow. They do *not* address the following remaining failure modes intrinsic to the TNNP cell model:

- Removable singularities in the α/β rate expressions of i_{Na} and i_{Kr} (notably the $1/(\exp(\cdot) - 1)$ factors at $V = -47.13$ mV and similar). These can be regularised *analytically* by L’Hôpital substitution near the singular point; the present wrapper leaves them to the adaptive integrator.
- Stiff blow-ups in the calcium-handling subsystem when Ca_{SR} approaches the floor or when fCa saturates close to 1. Clamping Ca_{SR} avoids overflow but does not regularise the Ca_i/Ca_{SR} ratio that drives the SR-release flux.
- Exponential overflow of the α/β tails for $|V| \gg \text{vmMax}$. Clamping V to the rate window is a workaround; the true fix is to ensure V never leaves the physiological window in the first place (i.e. pick a CFL/step combination that keeps the upstroke under control).

In short: the guard turns a numerical disaster (NaN cascade) into a recoverable mis-evaluation. It is not a substitute for an adequately-resolved outer solver.

7.6 Logging and accounting

Each invocation of `correctStateComponent` records a triplet (`phase`, `variable`, `reason`) together with the offending value, its corrected value, and the simulation time. If `TNNPLogNumericalProtection` is true, the entries are accumulated in memory (capped by `TNNPMaxProtectionLogEntries`) and emitted on solver exit to the file

`TNNP_numericalProtection_summary.dat`

in the case directory. The summary groups corrections by the (`phase`, `variable`, `reason`) key, with: count, first occurrence time, the first old/corrected value pair, and the min/max/ $|\Delta|_{\max}$ over all corrections in the group. Typical diagnostic use:

- Many `Vm_below_minimum` at one specific cell ID $\Rightarrow$ suspect a Dirichlet BC pulling V outside the rate window.
- Many `concentration_below_floor` after a long simulation $\Rightarrow$ ODE tolerances too loose; tighten `relTol`.
- Many `phase=derivatives` entries during the upstroke $\Rightarrow$ the inner ODE integrator is straining against the explicit voltage update; consider tightening the CFL coefficient or shrinking the requested Δt below Δt_{CFL} to recover head-room.
- Any `nonfinite_V` or `nonfinite_gate` $\Rightarrow$ a real divergence already happened upstream; the guard contained the damage but the result is suspect.

7.7 Defaults vs. recommended settings

The defaults are intentionally conservative (master switch `false`) to keep the wrapper transparent in test configurations. For production and convergence studies the recommended setting is:

Use case	TNNPNumericalProtection	Notes
Verification / MMS	<code>false</code>	Avoids hiding solver bugs.
Production runs	<code>true</code>	Gate/ V clamps on, log on.
Stress / stability	<code>true</code>	Inspect the summary file as the primary signal.

All 2D and 3D results in Section 10 were produced with the master switch `true`; each result directory contains the corresponding `TNNP_numericalProtection_summary.dat`.

8 Activation tracking and benchmarks

Activation time per cell. First-crossing of V_{thr} in a step is recorded by linear interpolation:

$$w = \text{clip}\left(\frac{V_{\text{thr}} - V^n}{V^{n+1} - V^n}, 0, 1\right), \quad t_{\text{act}} = t^n + w \Delta t. \quad (15)$$

The per-cell flag is cleared so re-crossings are ignored. Default threshold $V_{\text{thr}} = 0$ V (= 0 mV; the Niederer convention).

Early termination on P8. When `stopAfterPointActivation` = `true`, the cell nearest `stopActivationPoint` (P8 in the Niederer benchmark) is monitored, and the effective end time is set to $t_{\text{act}} + \text{stopDelayAfterActivation}$.

Diagonal sampling. `sampleIDW` performs inverse-distance-weighted interpolation at `nDiagonalSamples` points between `diagonalStart` and `diagonalEnd` with $k = \text{sampleNeighbours}$, $p = \text{samplePower}$.

9 Driver script: run_convergence.py

9.1 Geometry and parameter sweep

Python constant	OpenFOAM key	Meaning
DIMENSIONS	—	List of dimensions to sweep (2D, 3D).
DX_VALUES_MM	blockMeshDict cell counts	Uniform cell size in millimetres.
DT_VALUES	controlDict.deltaT	Requested outer time-step sizes (seconds).
END_TIME	controlDict.endTime	End time (s).
CFL	timeIntegrationProperties.CFL	Parabolic CFL safety coefficient (Eq. (6)).
ACTIVE_HO_PAIRS_Vm_States	useHighOrder_Vm useHighOrder_Iion LRECoeffs_*	/ Tuples (Vm-level, Iion-level); + level $\in \{N0, p1, p2, p3\}$.
ACTIVE_STATE_ODE_SOLVERS	timeIntegrationProperties.odeSolvers	ODE integrator names (RKF45, ...).
SIGMA_X, SIGMA_Y, SIGMA_Z	cardiacProperties	Diagonal of $\mathbf{D}$.
CHI	cardiacProperties.chi	Surface-to-volume ratio.
CM	cardiacProperties.Cm	Membrane capacitance.

9.2 Inner ODE integrator (RKF45)

Python constant	OpenFOAM key	Meaning
ODE_ABS_TOL	timeIntegrationProperties.absoluteTol	Absolute tolerance per state.
ODE_REL_TOL	timeIntegrationProperties.relativeTol	Relative tolerance per state.
ODE_INITIAL_STEP	initialODEStep	Initial sub-step (ms).
ODE_MAX_STEPS	maxSteps	RKF45 sub-step cap.

9.3 Effective Δt reported in result tables

Column	Meaning
dt_requested	Value of controlDict.deltaT written by the driver (= DT_VALUES).
dt_expectedSolver	$\Delta t = \min(\Delta t_{\text{req}}, \Delta t_{\text{CFL}})$ reconstructed by the driver from Eq. (6)–(7); this is the value the solver actually integrates with (it is also what runtime.setDeltaT stores in controlDict). When $\Delta t_{\text{req}} \geq \Delta t_{\text{CFL}}$ (the usual case in the reported sweeps), $\Delta t = \Delta t_{\text{CFL}}$.

9.4 Post-processing and run-control

Python constant	OpenFOAM key	Meaning
ACTIVATION_THRESHOLD	activationThreshold	V_{thr} (V). Default 0.
N_DIAGONAL_SAMPLES	nDiagonalSamples	IDW samples.
SAMPLE_NEIGHBOURS	sampleNeighbours	k -NN for IDW.
SAMPLE_POWER	samplePower	Inverse-distance power.
STOP_AFTER_POINT_ACTIVATION	stopAfterPointActivation	Early-stop on P8.
STOP_DELAY_AFTER_ACTIVATION	stopDelayAfterActivation	Delay (s).

Python constant	OpenFOAM key	Meaning
SAVE_LOGS_ON_SUCCESS	—	Python-only; keep logs after successful run.

10 Niederer-style benchmark results

This section collects measured P8 activation times, wall-clock cost and peak memory.

Scope and disclaimer. The published Niederer et al. 2012 [2] benchmark is three-dimensional ($20 \times 7 \times 3$ mm anisotropic slab with fibre rotation). The 2D case used here is a planar reduction (20×8 mm, single cell in z , planar fibres) defined for code-development testing only and not reflected, to our knowledge, in any published reference. The 3D case follows the Niederer slab geometry ($20 \times 7 \times 3$ mm) but the sweep is sparser (fewer LRE pairs and mesh sizes) than the 2D one. The numbers below should therefore be read as an *internal* self-consistency study (h - and Δt -refinement, plus the effect of the LRE p -levels) and not as a comparison with any external reference value.

Rows of the tables below are LRE p -levels for $V_m \times I_{\text{ion}}$ (NO = high-order disabled); columns are mesh sizes Δx in millimetres. Computational cost is the elapsed wall-clock time of the solver process reported by `/usr/bin/time -v`; peak memory is the maximum resident set size of the same process. NC = no run data found for that combination.

Reading Δt_{req} versus Δt_{CFL} . The forward-Euler scheme is bounded by the parabolic CFL condition (Section 3.1). The 2D sweep was driven with a single $\Delta t_{\text{req}} = 10^{-2}$ s; this value is much larger than the CFL bound at all mesh sizes used, so the solver always integrates with $\Delta t = \Delta t_{\text{CFL}}$ — the values are listed in the header table of §10.1. In the 3D sweep Δt_{req} is set close to the CFL bound, so for some configurations the CFL clamp *does not* bind and the solver runs at the requested step; for others it does. Both columns are reported.

10.1 2D Niederer-style sweep

The 2D sweep covers all 16 LRE pairs $(\text{NO}, p1, p2, p3) \times (\text{NO}, p1, p2, p3)$ at five mesh sizes $\Delta x \in \{0.5, 0.25, 0.125, 0.1, 0.05\}$ mm, with a single $\Delta t_{\text{req}} = 10^{-2}$ s.

10.2 P8 activation time at the requested $\Delta t = 1.0 \cdot 10^{-2}$ s

Effective time step set by the explicit CFL limit:

Δx [mm]	0.5	0.25	0.125	0.1	0.05
Δt_{eff} [s]	$1.3 \cdot 10^{-4}$	$3.3 \cdot 10^{-5}$	$8.2 \cdot 10^{-6}$	$5.2 \cdot 10^{-6}$	$1.3 \cdot 10^{-6}$

Table 5: P8 activation time [ms]: rows are LRE p -levels for $V_m \times I_{\text{ion}}$; columns are mesh sizes Δx (mm). NC = no run data found.

V_m	I_{ion}	0.5	0.25	0.125	0.1	0.05
NO	NO	160.63	68.12	50.35	48.15	45.62
NO	p1	96.43	59.69	48.33	47.08	45.53
NO	p2	34.71	42.82	44.70	44.95	NC
NO	p3	35.33	43.46	44.67	44.92	NC
p1	NO	14.25	75.16	54.23	51.20	46.59
p1	p1	104.08	65.69	51.94	49.91	46.47
p1	p2	38.45	47.47	47.24	46.97	46.02
p1	p3	40.65	48.30	47.22	46.94	NC

Vm	Iion	0.5	0.25	0.125	0.1	0.05
p2	NO	0.00	89.11	60.13	55.60	47.85
p2	p1	159.22	74.08	56.36	53.33	47.58
p2	p2	42.88	48.89	48.50	48.14	46.56
p2	p3	48.72	50.35	48.65	48.19	NC
p3	NO	192.55	73.84	54.51	51.31	46.24
p3	p1	67.83	62.86	51.48	49.52	46.07
p3	p2	34.76	43.10	45.51	45.70	NC
p3	p3	36.02	43.80	45.54	45.69	NC

10.3 Wall-clock cost per configuration (minutes)

Total solver elapsed time as reported by `/usr/bin/time -v`.

Table 6: Solver wall-clock time [min].

Vm	Iion	0.5	0.25	0.125	0.1	0.05
NO	NO	1	2	9	14	168
NO	p1	1	3	15	25	416
NO	p2	1	6	42	73	NC
NO	p3	1	9	59	100	NC
p1	NO	1	2	9	15	203
p1	p1	1	3	17	27	381
p1	p2	1	7	45	77	1059
p1	p3	2	9	60	108	NC
p2	NO	1	2	10	16	239
p2	p1	1	4	19	31	398
p2	p2	1	7	47	77	1167
p2	p3	2	10	62	113	NC
p3	NO	1	2	9	15	197
p3	p1	1	3	18	31	402
p3	p2	1	6	47	75	NC
p3	p3	1	9	58	106	NC

10.4 Peak resident set size per configuration (MiB)

Maximum physical RAM used by the solver process.

Table 7: Peak RSS [MiB].

Vm	Iion	0.5	0.25	0.125	0.1	0.05
NO	NO	148	152	165	179	297
NO	p1	149	155	182	202	388
NO	p2	152	168	234	287	NC
NO	p3	154	174	261	329	NC
p1	NO	148	154	177	198	377
p1	p1	150	158	200	234	516
p1	p2	153	170	254	318	855
p1	p3	155	176	282	361	NC
p2	NO	149	155	180	204	398
p2	p1	150	159	217	260	622
p2	p2	153	173	271	345	961
p2	p3	155	180	298	388	NC

Vm	Iion	0.5	0.25	0.125	0.1	0.05
p3	N0	150	160	202	236	524
p3	p1	151	174	277	354	995
p3	p2	155	188	331	439	NC
p3	p3	156	195	356	482	NC

10.4.1 Discussion of the 2D results

***h*-refinement at fixed configuration.** For the low-order pair (N0, N0), the P8 activation time decreases monotonically with mesh refinement ($160.63 \rightarrow 68.12 \rightarrow 50.35 \rightarrow 48.15 \rightarrow 45.62$ ms from $\Delta x = 0.5$ to 0.05 mm). At each Δx the effective step also decreases as Δx^2 ($1.3 \cdot 10^{-4} \rightarrow 1.3 \cdot 10^{-6}$ s); the forward-Euler scheme is therefore well below its stability bound *by construction* on every line of the table. The asymptotic value at the finest mesh is ≈ 45.6 ms.

High-order on the source. Activating the LRE quadrature on I_{ion} — without changing the diffusion (N0, p2) — drops the coarsest-mesh value from 160.63 ms to 34.71 ms and brings the finer-mesh values to ≈ 45 ms already at $\Delta x = 0.125$. The high-order ionic quadrature is therefore the most cost-effective single switch for accuracy on coarse meshes; see Eq. (14) for the mechanism.

High-order on the diffusion alone. The combinations (p1, N0), (p2, N0), (p3, N0) — high-order LRE diffusion with cell-centre ionic source — are noticeably less stable on the coarsest mesh: p1/N0 at $\Delta x = 0.5$ mm returns 14.25 ms (clearly a spurious very-early crossing), and p2/N0 returns 0.00 ms (the front never crossed within the simulated window, or crossed spuriously at $t = 0$). These values are diagnostic, not physical: the high-order Laplacian operating on a piecewise-constant ionic source produces an over-shooting voltage on coarsely-resolved fronts, which the activation-time tracker then latches at the first numerical crossing. The pathology disappears once the ionic source is also volume-integrated at LRE quadrature points (any p_k/p_k pair gives a well-behaved value on the same mesh).

Full HO at the finer meshes. The diagonal pairs (p2, p2) and (p3, p3) plateau quickly: at $\Delta x = 0.125$ mm and $\Delta x = 0.1$ mm the values are 48.5/48.1 ms and 45.5/45.7 ms respectively; at $\Delta x = 0.05$ mm the (p2, p2) value is 46.6 ms. The off-diagonal pairs (N0, p2), (N0, p3), (p1, p2), (p1, p3), (p3, p2), (p3, p3) all collapse to a mutually-consistent value in $[44.5, 46.0]$ ms at $\Delta x \leq 0.1$ mm. This $[44.5, 46.0]$ ms band is the internal-consistency floor of the 2D dataset; we do not claim it represents the “true” value for this 2D problem.

Cost. Wall-clock cost is dominated by the $N_c \times$ (ionic-ODE work per cell) product and by the inverse Δt_{CFL} . Refining the mesh from $\Delta x = 0.5$ to 0.05 mm increases the cell count by 10^2 and the step count by $\sim 10^2$, so the total cost grows by $\sim 10^4$ — consistent with the (N0, N0) entries of the wall-clock table (1 min vs. 168 min). Peak RSS grows roughly linearly with N_c , with HO levels adding a constant-factor overhead for the LRE stencils and the per-quadrature-point state vectors.

10.5 3D Niederer-style sweep

The 3D sweep is much sparser: eight LRE pairs $\{(\text{N0}, \text{N0}), (\text{N0}, \text{p1}), (\text{N0}, \text{p2}), (\text{p1}, \text{p1}), (\text{p2}, \text{p1}), (\text{p2}, \text{p2}), (\text{p3}, \text{p1}), (\text{p3}, \text{p3})\}$, three requested time steps $\Delta t_{\text{req}} \in \{1 \cdot 10^{-5}, 5 \cdot 10^{-5}, 5 \cdot 10^{-6}\}$ s, and mesh sizes $\Delta x \in \{0.5, 0.25, 0.2\}$ mm covering a $20 \times 3 \times 7$ mm slab (Niederer geometry, $40 \times 6 \times 14$ to $100 \times 15 \times 35$ cells). Most combinations exist only for $\Delta x = 0.5$ mm; one mesh refinement to $\Delta x = 0.25$ mm exists for half the LRE pairs at $\Delta t_{\text{req}} = 5e-5$; $\Delta x = 0.2$ mm exists only for (N0, N0) and (N0, p1) at $\Delta t_{\text{req}} = 5e-5$.

10.6 P8 activation time at the requested Δt

Effective time step set by the explicit CFL limit:

Δx [mm]	0.5	0.25	0.2	0.1
Δt_{eff} [s]	$8.7 \cdot 10^{-5}$	$2.1 \cdot 10^{-5}$	$1.3 \cdot 10^{-5}$	$3.4 \cdot 10^{-6}$

Table 8: P8 activation time [ms]: rows are LRE p-levels for $V_m \times I_{\text{ion}}$; columns are mesh sizes Δx (mm). NC = no run data found.

V_m	I_{ion}	Δt_{req} [s]	0.5	0.25	0.2	0.1
NO	NO	$5 \cdot 10^{-5}$	141.52	61.68	54.32	45.26
NO	NO	$1 \cdot 10^{-5}$	140.96	NC	NC	NC
NO	NO	$5 \cdot 10^{-6}$	140.91	NC	NC	NC
NO	p1	$5 \cdot 10^{-5}$	36.35	42.25	42.79	NC
NO	p1	$1 \cdot 10^{-5}$	36.66	NC	NC	NC
NO	p1	$5 \cdot 10^{-6}$	36.71	NC	NC	NC
p1	p1	$5 \cdot 10^{-5}$	38.75	45.44	NC	NC
p1	p1	$1 \cdot 10^{-5}$	39.10	NC	NC	NC
p1	p1	$5 \cdot 10^{-6}$	39.16	NC	NC	NC
p2	p1	$5 \cdot 10^{-5}$	41.13	46.41	NC	NC
p2	p1	$1 \cdot 10^{-5}$	41.46	NC	NC	NC
p2	p1	$5 \cdot 10^{-6}$	41.53	NC	NC	NC
p2	p2	$5 \cdot 10^{-5}$	35.56	42.80	NC	NC
p2	p2	$1 \cdot 10^{-5}$	35.83	NC	NC	NC
p2	p2	$5 \cdot 10^{-6}$	35.89	NC	NC	NC
p3	p1	$5 \cdot 10^{-5}$	35.51	42.14	NC	NC
p3	p1	$1 \cdot 10^{-5}$	35.85	NC	NC	NC
p3	p1	$5 \cdot 10^{-6}$	35.92	NC	NC	NC
p3	p3	$5 \cdot 10^{-5}$	32.64	40.27	NC	NC
p3	p3	$1 \cdot 10^{-5}$	32.94	NC	NC	NC
p3	p3	$5 \cdot 10^{-6}$	33.00	NC	NC	NC

10.7 Wall-clock cost per configuration (minutes)

Total solver elapsed time as reported by `/usr/bin/time -v`.

Table 9: Solver wall-clock time [min].

V_m	I_{ion}	Δt_{req} [s]	0.5	0.25	0.2	0.1
NO	NO	$5 \cdot 10^{-5}$	3	16	40	384
NO	NO	$1 \cdot 10^{-5}$	6	NC	NC	NC
NO	NO	$5 \cdot 10^{-6}$	9	NC	NC	NC
NO	p1	$5 \cdot 10^{-5}$	11	137	288	NC
NO	p1	$1 \cdot 10^{-5}$	19	NC	NC	NC
NO	p1	$5 \cdot 10^{-6}$	27	NC	NC	NC
p1	p1	$5 \cdot 10^{-5}$	12	158	NC	NC
p1	p1	$1 \cdot 10^{-5}$	20	NC	NC	NC
p1	p1	$5 \cdot 10^{-6}$	29	NC	NC	NC
p2	p1	$5 \cdot 10^{-5}$	13	166	NC	NC
p2	p1	$1 \cdot 10^{-5}$	21	NC	NC	NC
p2	p1	$5 \cdot 10^{-6}$	31	NC	NC	NC
p2	p2	$5 \cdot 10^{-5}$	52	632	NC	NC
p2	p2	$1 \cdot 10^{-5}$	81	NC	NC	NC
p2	p2	$5 \cdot 10^{-6}$	126	NC	NC	NC

Vm	lion	$\Delta t_{\text{req}} [\text{s}]$	0.5	0.25	0.2	0.1
p3	p1	$5 \cdot 10^{-5}$	12	150	NC	NC
p3	p1	$1 \cdot 10^{-5}$	20	NC	NC	NC
p3	p1	$5 \cdot 10^{-6}$	31	NC	NC	NC
p3	p3	$5 \cdot 10^{-5}$	102	1416	NC	NC
p3	p3	$1 \cdot 10^{-5}$	168	NC	NC	NC
p3	p3	$5 \cdot 10^{-6}$	297	NC	NC	NC

10.8 Peak resident set size per configuration (MiB)

Maximum physical RAM used by the solver process.

Table 10: Peak RSS [MiB].

Vm	lion	$\Delta t_{\text{req}} [\text{s}]$	0.5	0.25	0.2	0.1
NO	NO	$5 \cdot 10^{-5}$	153	210	278	1234
NO	NO	$1 \cdot 10^{-5}$	153	NC	NC	NC
NO	NO	$5 \cdot 10^{-6}$	153	NC	NC	NC
NO	p1	$5 \cdot 10^{-5}$	199	609	1054	NC
NO	p1	$1 \cdot 10^{-5}$	199	NC	NC	NC
NO	p1	$5 \cdot 10^{-6}$	199	NC	NC	NC
p1	p1	$5 \cdot 10^{-5}$	219	764	NC	NC
p1	p1	$1 \cdot 10^{-5}$	219	NC	NC	NC
p1	p1	$5 \cdot 10^{-6}$	219	NC	NC	NC
p2	p1	$5 \cdot 10^{-5}$	245	976	NC	NC
p2	p1	$1 \cdot 10^{-5}$	245	NC	NC	NC
p2	p1	$5 \cdot 10^{-6}$	245	NC	NC	NC
p2	p2	$5 \cdot 10^{-5}$	405	2262	NC	NC
p2	p2	$1 \cdot 10^{-5}$	405	NC	NC	NC
p2	p2	$5 \cdot 10^{-6}$	405	NC	NC	NC
p3	p1	$5 \cdot 10^{-5}$	381	2056	NC	NC
p3	p1	$1 \cdot 10^{-5}$	381	NC	NC	NC
p3	p1	$5 \cdot 10^{-6}$	381	NC	NC	NC
p3	p3	$5 \cdot 10^{-5}$	862	5909	NC	NC
p3	p3	$1 \cdot 10^{-5}$	862	NC	NC	NC
p3	p3	$5 \cdot 10^{-6}$	862	NC	NC	NC

10.8.1 Discussion of the 3D results

Δt_{req} versus Δt_{CFL} . In the 3D sweep some rows have $\Delta t_{\text{req}} = \Delta t_{\text{CFL}}$ (the requested step happened to be at or below the CFL bound, so the solver integrated with that value), while others have $\Delta t_{\text{req}} > \Delta t_{\text{CFL}}$ (the CFL clamp binds, the solver integrates with the smaller CFL value). Both columns are reported because they are dictated by different constraints: Δt_{req} comes from the user’s `controlDict`, Δt_{CFL} from the mesh-and-conductivity product.

Coarse-mesh sensitivity. At the coarsest mesh ($\Delta x = 0.5$ mm, $40 \times 6 \times 14$ cells), the low-order (NO, NO) configuration gives $t_{\text{act}}^{P8} \approx 141$ ms — consistent across both $\Delta t = 10^{-5}$ and $5 \cdot 10^{-6}$ s (140.96 vs. 140.91 ms): the temporal error is small but the spatial error is dominant. Adding even p1 to the ionic quadrature collapses the value to ≈ 36.7 ms at the same mesh, with the same temporal stability. The (p3, p3) pair reaches ≈ 33 ms on the same coarse mesh.

One mesh refinement. For the rows that exist at $\Delta x = 0.25$ mm (with $\Delta t_{\text{CFL}} \approx 2.2 \cdot 10^{-5}$ s), the high-order pairs cluster in $[40.3, 46.4]$ ms across the four LRE combinations available. The (p3, p3) pair value 40.27 ms is the lowest of this group.

Where the 2D and 3D values disagree. The 2D and 3D problems are different (different geometry, different ratio of diffusion to reaction), so the activation times are not comparable directly. The qualitative pattern is the same in both cases: high-order on the source (`pk`, second slot) is the single biggest accuracy lever; mesh-only refinement with low-order diffusion converges from above (the front arrives later than the asymptotic value); turning on high-order diffusion in isolation (`pk`, `N0`) is unstable on coarse meshes.

10.9 Conclusions on the 2D/3D study

1. The forward-Euler scheme is always run at $\Delta t = \Delta t_{\text{CFL}}$ when $\Delta t_{\text{req}} \gg \Delta t_{\text{CFL}}$ (the 2D sweep) and at $\Delta t = \Delta t_{\text{req}}$ otherwise (some of the 3D rows). The two columns in the result tables are not redundant; they reflect a `min()` clamp in the solver and the actually-used step is the smaller of the two.
2. The LRE quadrature on the ionic source (`useHighOrder_Iion`) is the most cost-effective accuracy switch on both 2D and 3D meshes; on coarse meshes it brings the P8 activation time from ~ 160 ms (2D) / ~ 140 ms (3D) to physical values ~ 40 ms.
3. The high-order LRE diffusion in isolation (`useHighOrder_Vm` with cell-centre ionic source) is *not* robust on coarse meshes; the high-order reconstruction of the Laplacian on top of a piecewise-constant source leads to non-physical early crossings. The recommended use is to enable both switches together, or to keep the low-order branch on the diffusion when the source is also low-order.
4. The 2D internal-consistency floor (best mutually-consistent band across high-order pairs at the finer meshes) sits at $[44.5, 46.0]$ ms; the 3D high-order pairs at $\Delta x = 0.25$ mm sit in $[40.3, 46.4]$ ms. We do not claim either band represents the “true” value for the respective problem; published 3D Niederer values are reported with a wide cross-code spread.
5. For matched accuracy, the explicit forward-Euler scheme is much more expensive than the implicit Crank-Nicolson scheme of the companion solver, because $\Delta t_{\text{CFL}} = \mathcal{O}(\Delta x^2)$ binds and the per-step cost is dominated by the cell-model ODE work (which is independent of the time discretisation). The explicit solver is competitive on coarse meshes and as a verification baseline for the implicit solver, not as a production tool on fine meshes.

A Dimension table

Field	OpenFOAM dimensionSet	SI
<code>Vm</code>	$[1, 2, -3, 0, 0, -1, 0]$	volt
<code>Iion</code>	<code>dimVoltage/dimTime</code>	V s^{-1}
<code>externalStimulusCurrent</code>	<code>dimCurrent/dimVolume</code>	A m^{-3}
<code>conductivity</code>	<code>dimCurrent^2 dimTime^3 / (dimMass dimVolume)</code>	S m^{-1}
<code>chi</code>	<code>dimless/dimLength</code>	m^{-1}
<code>Cm</code>	<code>dimCurrent dimTime / (dimVoltage dimArea)</code>	F m^{-2}

B TNNP rate expressions (verbatim)

For completeness, the rate expressions used by the TNNP 2004 cell model wrapper are reproduced below. The membrane voltage entering these expressions is in millivolts ($V = 10^3 V_m$ when the PDE-side voltage is in volts); concentrations are in mM; time constants are in ms. References are to [1].

Reversal potentials.

$$E_{Na} = \frac{RT}{F} \ln \left(\frac{Na_o}{Na_i} \right), \quad (16)$$

$$E_K = \frac{RT}{F} \ln \left(\frac{K_o}{K_i} \right), \quad (17)$$

$$E_{Ks} = \frac{RT}{F} \ln \left(\frac{K_o + P_{kna} Na_o}{K_i + P_{kna} Na_i} \right), \quad (18)$$

$$E_{Ca} = \frac{1}{2} \frac{RT}{F} \ln \left(\frac{Ca_o}{Ca_i} \right). \quad (19)$$

Total ionic current.

$$I_{ion} = i_{K1} + i_{to} + i_{Kr} + i_{Ks} + i_{CaL} + i_{NaK} + i_{Na} + i_{bNa} + i_{NaCa} + i_{bCa} + i_{pK} + i_{pCa}. \quad (20)$$

Individual currents.

$$i_{Na} = g_{Na} m^3 h j (V - E_{Na}), \quad (21)$$

$$i_{bNa} = g_{bNa} (V - E_{Na}), \quad (22)$$

$$i_{K1} = g_{K1} x_{K1,\infty} \sqrt{K_o/5.4} (V - E_K), \quad (23)$$

$$i_{to} = g_{to} r s (V - E_K), \quad (24)$$

$$i_{Kr} = g_{Kr} \sqrt{K_o/5.4} X_{r1} X_{r2} (V - E_K), \quad (25)$$

$$i_{Ks} = g_{Ks} X_s^2 (V - E_{Ks}), \quad (26)$$

$$i_{bCa} = g_{bCa} (V - E_{Ca}), \quad (27)$$

$$i_{pK} = \frac{g_{pK} (V - E_K)}{1 + \exp((25 - V)/5.98)}, \quad (28)$$

$$i_{pCa} = \frac{g_{pCa} Ca_i}{Ca_i + K_{pCa}}. \quad (29)$$

$$i_{CaL} = \frac{g_{CaL} d f f_{Ca} \frac{4VF^2}{RT} (Ca_i \exp(2VF/RT) - 0.341 Ca_o)}{\exp(2VF/RT) - 1}. \quad (30)$$

$$i_{NaK} = \frac{P_{NaK} \frac{K_o}{K_o + K_{mk}} \frac{Na_i}{Na_i + K_{mNa}}}{1 + 0.1245 \exp(-0.1VF/RT) + 0.0353 \exp(-VF/RT)}, \quad (31)$$

$$i_{NaCa} = \frac{K_{NaCa} [\exp(\gamma VF/RT) Na_i^3 Ca_o - \exp((\gamma - 1)VF/RT) Na_o^3 Ca_i \alpha]}{(K_{mNa}^3 + Na_o^3)(K_{mCa} + Ca_o) [1 + K_{sat} \exp((\gamma - 1)VF/RT)]}. \quad (32)$$

Gate kinetics (selected; full set generated by the TNNP code generator). Most gates use the standard x_∞/τ_x form $\dot{x} = (x_\infty - x)/\tau_x$. The implemented steady states and time constants for $m, h, j, d, f, X_{r1}, X_{r2}, X_s, r$ are the ones of [1] and are not reproduced here in full. The s gate depends on tissue type:

$$s_\infty = \begin{cases} [1 + \exp((V + 20)/5)]^{-1}, & \text{endocardial,} \\ 1.1 [1 + \exp((V + 28)/6)]^{-1}, & \text{M cell or epicardial,} \end{cases} \quad (33)$$

with τ_s also tissue-dependent. The calcium-dependent gates follow

$$\alpha_{fCa} = [1 + (Ca_i/0.000325)^8]^{-1}, \quad (34)$$

$$\beta_{fCa} = 0.1 [1 + \exp((Ca_i - 0.0005)/0.0001)]^{-1}, \quad (35)$$

$$\beta_d = 0.2 [1 + \exp((Ca_i - 0.00075)/0.0008)]^{-1}, \quad (36)$$

$$f_{Ca,\infty} = (\alpha_{fCa} + \beta_{fCa} + \beta_d + 0.23)/1.46, \quad (37)$$

with the conditionally-frozen update

$$\frac{df_{Ca}}{dt} = \begin{cases} 0, & f_{Ca,\infty} > f_{Ca} \text{ and } V > -60, \\ (f_{Ca,\infty} - f_{Ca})/\tau_{fCa}, & \text{otherwise,} \end{cases} \quad (38)$$

and analogously for the RyR g gate.

Concentrations and SR fluxes.

$$\frac{dNa_i}{dt} = -\frac{(i_{Na} + i_{bNa} + 3i_{NaK} + 3i_{NaCa})C_m}{V_c F}, \quad (39)$$

$$\frac{dK_i}{dt} = -\frac{[(i_{K1} + i_{to} + i_{Kr} + i_{Ks} + i_{pK} + I_{stim}) - 2i_{NaK}]C_m}{V_c F}, \quad (40)$$

$$i_{rel} = \left[\frac{a_{rel}Ca_{SR}^2}{b_{rel}^2 + Ca_{SR}^2} + c_{rel} \right] dg, \quad (41)$$

$$i_{up} = \frac{V_{max,up}}{1 + K_{up}^2/Ca_i^2}, \quad (42)$$

$$i_{leak} = V_{leak}(Ca_{SR} - Ca_i), \quad (43)$$

$$J_{Ca_i} = -\frac{(i_{CaL} + i_{bCa} + i_{pCa} - 2i_{NaCa})C_m}{2V_c F} + i_{leak} - i_{up} + i_{rel}, \quad (44)$$

$$\frac{dCa_i}{dt} = J_{Ca_i} f_{JCa_i}, \quad (45)$$

$$\frac{dCa_{SR}}{dt} = \frac{V_c}{V_{SR}}(i_{up} - i_{rel} - i_{leak}) f_{JCa_{SR}}, \quad (46)$$

with buffering factors $f_{JCa_i}, f_{JCa_{SR}}$ of the form $[1 + \text{Buf} \cdot K/(Ca + K)^2]^{-1}$.

References

- [1] K. H. W. J. ten Tusscher, D. Noble, P. J. Noble, A. V. Panfilov, *A model for human ventricular tissue*, AJP – Heart and Circulatory Physiology **286** (2004) H1573–H1589.
- [2] S. A. Niederer et al., *Verification of cardiac tissue electrophysiology simulators using an N-version benchmark*, Prog. Biophys. Mol. Biol. **104** (2012) 22–48.
- [3] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II*, Springer, 1996.
- [4] P. Cardiff, A. Karač, P. De Jaeger, H. Jasak, J. Nagy, A. Ivanković, Ž. Tuković, *An open-source finite volume toolbox for solid mechanics and fluid–solid interaction simulations*, arXiv:1808.10736 (2018). Provides the LRE high-order reconstruction library (LRE.H) used in §4.2.